CHAPTER 1

ALGORITHMS AND FLOWCHARTS

C++ Programming • Chapter 1

Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology



Describe the solution before translating it into C++ code.

01

ALGORITHMS

A step-by-step method transforms data into a desired result.

Learning outcomes

By the end of this chapter, you can...

1. **Identify the problem data**
   Distinguish inputs, processing steps, and outputs.

2. **Represent an algorithm**
   Describe the same solution with a flowchart or pseudocode.

3. **Trace control flow**
   Follow sequence, selection, and repetition step by step.

4. **Translate the logic**
   Turn a simple algorithm into a basic C++ program.

Focus on measurable actions: identify, represent, trace, and translate.

A four-part learning path

Move from problem analysis to executable logic.

Each stage answers one practical question about the solution.



Analyze

Input • Processing • Output

Represent

Flowchart • Pseudocode

Control

Sequence • Selection • Repetition

Translate

Corresponding C++ code

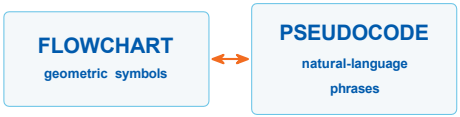
One solution model connects every example in the chapter.

An algorithm states how a problem will be solved

It is a sequence of step-by-step instructions.

An algorithm describes how data are processed to produce the desired result.

It answers: “What method will be used to solve the problem?”



Same algorithm, two representations.

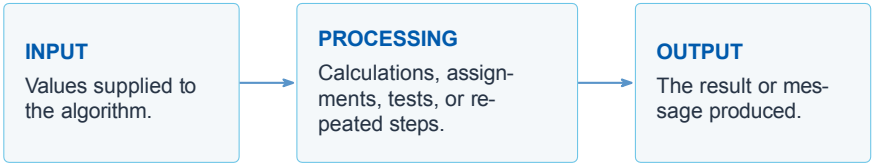
01 Visual
See the overall structure.

02 Editable
Easy to write and modify.

03 Portable
Translate into C++, Java, or Python.

Start every example with data analysis

Separate what enters, what changes, and what leaves.



Employee pay: Name, Hours, Rate $\longrightarrow$ Pay = Hours $\times$ Rate $\longrightarrow$ Name, Pay

02

FLOWCHARTS

A small visual vocabulary makes execution order visible.

Seven standard symbols form the flowchart vocabulary

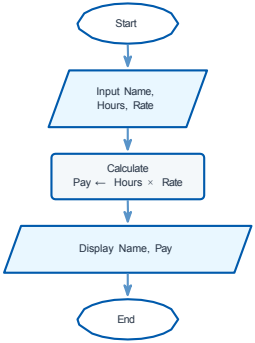
Use each shape consistently and connect it with directed arrows.

Shape	Symbol	Purpose
	**Oval — Start/End (Terminal)**	Marks the starting or ending point.
	**Parallelogram — Input/Output**	Reads data from an input device or writes data to an output device.
	**Rectangle — Process**	An assignment of a value or a computational operation.
	**Diamond — Decision**	A true/false question with true and false branches.
	Arrow — Flowline	Indicates the execution direction between steps.
	**Small circle — Connector**	Connects separated parts of a flowchart on the same page and reduces crossing flow-lines.
	**Rectangle with double vertical edges** **Predefined process**	Calls an algorithm, function, or sub-process defined elsewhere.

Label decision branches clearly: Yes/No or True/False.

Example 1: calculating an employee's pay

A sequence executes from top to bottom.



Data analysis

- Input: Name, Hours, Rate
- Processing: Pay ← Hours × Rate
- Output: Name, Pay

Arrows show execution order. There are no branches and no loops.

Pseudocode and C++ express the same algorithm

Employee pay: the solution logic remains unchanged.

Pseudocode

```
INPUT Name, Hours, Rate
CALCULATE Pay = Hours * Rate
DISPLAY Name, Pay
```

Three steps are performed in order: read, calculate, display.

Corresponding C++ code

```
string name;
double hours, rate, pay;

cout << "Name: "; getline(cin,
name); cout << "Hours: "; cin >>
hours;
cout << "Rate: "; cin >> rate;

pay = hours * rate;
cout << name << " earns " << pay <<
endl;
```

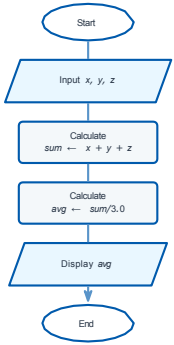
Example 2: arithmetic mean of three numbers

Two processing blocks make the arithmetic explicit.

Data analysis

- Input: *x*, *y*, *z*
- Processing: *sum* ← *x* + *y* + *z*
- Processing: *avg* ← *sum*/3.0
- Output: *avg*

```
INPUT x, y, z
CALCULATE sum = x + y + z
CALCULATE avg = sum / 3.0
DISPLAY avg
```



Dividing by 3.0 preserves the fractional part of the result.

03

PSEUDOCODE

A language-independent blueprint for program logic.

Three conventions keep pseudocode readable

The goal is clear logic, not programming-language syntax.

01 Keywords

Write control words in upper-case: INPUT, CALCULATE, PRINT, IF, ELSE, DO, WHILE.

02 Indentation

Indent statements to show the scope of a branch or loop.

03 Assignment

Use the form `Variable = Expression`.

Pseudocode is a linear text representation of the same algorithm shown by a flowchart.

Use it as a blueprint before writing the corresponding C++ code.

04

BRANCHING

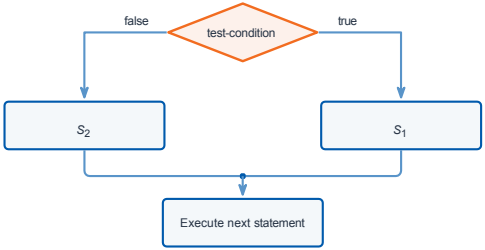
A condition chooses one of two or more execution paths.

IF–THEN–ELSE creates two possible paths

After either branch, execution continues with the next statement.

```
IF test-condition THEN
    S1
ELSE
    S2
ENDIF
```

```
IF test-condition THEN
    S
ENDIF
```



Example 3: square root or negative-number message

Calculate $B = \sqrt{A}$ when $A \geq 0$; otherwise report that A is negative.

Pseudocode

```
INPUT A
IF A >= 0 THEN
    CALCULATE B = sqrt(A)
    PRINT B
ELSE
    PRINT "A is negative"
ENDIF
```

Input: A Output: B or a message.

Corresponding

```
double A, B;
cin >> A;

if (A >= 0)
{
    B =
    sqrt(A);
    cout <<
    "B = "
    << B <<
    endl;
} else {
    cout <<
    "A is
    negative
    " <<
    endl;
}
```

The case $A = 0$ is valid because $\sqrt{0} = 0$.

Example 4: solving a quadratic equation

Solve $ax^2 + bx + c = 0$ for $a \neq 0$.

$$\Delta = b^2 - 4ac$$

$\Delta > 0$

Two distinct roots:

$$x_{1,2} = \frac{-b \pm \sqrt{\Delta}}{2a}$$

$\Delta = 0$

One repeated root:

$$x = \frac{-b}{2a}$$

$\Delta < 0$

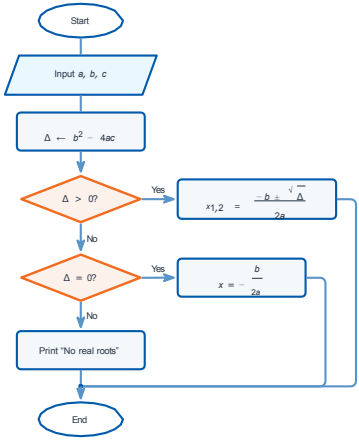
The equation has no real roots.

If $a = 0$, the equation becomes linear and must be handled separately.

Multi-branch conditions are checked in sequence

The quadratic-equation algorithm uses IF–ELSE IF–ELSE.

```
INPUT a, b, c
CALCULATE del = b^2 - 4*a*c
IF del > 0 THEN
    x1 = (-b +
    sqrt(del)) / (2*a)
    x2 = (-b - sqrt(del)) / (2*
    a)  PRINT x1, x2
ELSE IF del = 0 THEN
    x = -b / (2*a)
    PRINT x
ELSE
    PRINT "No real roots"
ENDIF
```



05

LOOPS

Repeat the same calculation or sequence of statements.

Continuation and exit conditions describe the same loop

The difference is the meaning assigned to a true result.

Pseudocode: continuation condition

WHILE (NUM <= 9)

When the condition is true, the loop continues.

Flowchart: exit condition

NUM > 9?

When the condition is true, the loop stops.

```
DO
    <statements in the loop body>
WHILE (condition)
```

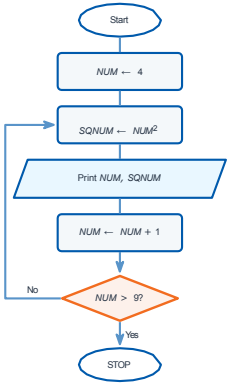
The two conditions are logically equivalent, but expressed from opposite viewpoints.

Example 5: numbers 4 to 9 and their squares

Initialize, process, advance, then test.

```
NUM = 4
DO
    SQNUM = NUM^2
    PRINT NUM, SQNUM
    NUM = NUM + 1
WHILE (NUM <= 9)
```

Output pairs: (*NUM*, *SQNUM*).



Example 5 translated into a do–while loop

The C++ condition matches the pseudocode continuation condition.

```
int num = 4;
do {
    int
    sqnum =
    num *
    num;
    cout << num << " " << sqnum << endl;
    num++;
} while (num <= 9);
```

Trace the state

Each iteration prints one pair, then replaces the old value of `num` with `num + 1`.

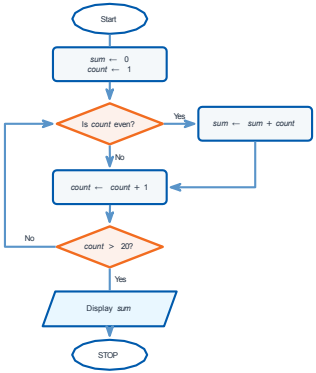
Stop condition

After printing 9 and 81, `num` becomes 10 and the loop ends.

Example 6: sum of the even numbers from 1 to 20

This example combines a branch with a loop.

```
sum = 0
count = 1
DO
    IF
    ( cou
    nt is
    even)
    THEN
        sum = sum + count
    ENDIF
    count = count + 1
WHILE (count <= 20)
DISPLAY sum
```



$2 + 4 + 6 + \cdots + 20 = 110$

Practice: draw a flowchart and write pseudocode

The exercises progress from sequence to branching and repetition.

1. Read two numbers *A* and *B*, then display their sum.
2. Read two numbers *A* and *B*, then display the smaller value.
3. Read three numbers *A*, *B*, and *C*, then display the smallest value.
4. Repeatedly read numbers until the user enters 0.
5. Repeatedly read numbers until the user enters 0; then display the sum of all entered numbers.

For every exercise, identify Input–Processing–Output before drawing the control flow.

Check your flowchart vocabulary

Choose the single best answer.

Which flowchart symbol represents a true/false question with two branches?

A. Oval

B. Rectangle

C. **Diamond**

D. Parallelogram

Answer: C. The diamond is the Decision symbol; label its branches clearly.

Key messages

Keep the conclusion short, ordered, and easy to recall.

- **Foundation:** An algorithm is a step-by-step method for producing a desired result.
- **Representation:** Flowcharts provide a visual overview; pseudocode details each step.
- **Control:** Sequence, selection, and repetition determine the execution flow.
- **Application:** Identify Input–Processing–Output, then translate the logic into C++.

Analyze first, represent the logic, then translate it into C++.

Chapter source

All instructional content comes from the supplied course document.

Course document

C++ Programming
Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology

Assigned reading

Chapter 1: Algorithms and Flowcharts
Sections 1.1–1.7 • pages 7–19

CHAPTER 1

Algorithms and Flowcharts

From problem statement to clear program logic.